MockMaster: AI-Based Mock Interview Evaluation System

Enrolment No: 23103180,23103179,23103153

Name of Students: Saksham Tripathi , Shubhankar Kumar , Aman Kumar Singh Name of

Supervisor: Prateek Kumar Soni

Submitted to – Mrs. Diksha Chawla
Mr. Rohit Sony

MAY- 2026

Submitted in partial fulfillment of the Degree of

Bachelor of Technology in Computer Science Engineering

MINOR PROJECT- 2

(Minor Project work summary sheet for Quick reference of Panel and Supervisor)

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY, SECTOR 62, NOIDA

Motivation behind the project

Technical interviews cause immense anxiety, and candidates often fail not due to a lack of technical knowledge, but due to poor communication or visible nervousness. Existing mock interview platforms either focus solely on coding logic or rely on expensive, subjective human coaches. There is a pressing need for an accessible, objective, and candidate-centric tool that utilizes AI to evaluate both technical accuracy and behavioral delivery.

Type of project

Development cum research project

Critical Analysis of research paper read and gaps in work and one line summary of each paper studied

- **Paper 1 Summary:** Explores the application of NLP (TF-IDF and cosine similarity) for grading short-text technical answers with human-level accuracy.
- **Paper 2 Summary:** Discusses real-time facial expression recognition and emotional classification in e-learning using lightweight Convolutional Neural Networks (CNNs).
- **Identified Gaps:** Current educational platforms focus purely on code execution (e.g., LeetCode), while advanced AI video screeners (e.g., HireVue) are built solely for recruiters to filter candidates. There is a lack of a unified platform that integrates behavioral tracking and semantic NLP evaluation directly for the candidate's personal improvement.

Overall design of project

The project follows a decoupled, microservices-inspired architecture:

- **Frontend (Client):** A Vite-powered Single Page Application (SPA) handling UI, state management, audio capture via Web Speech API, and local video processing using WebGL.
- **Backend (Serverless):** Node.js Express application deployed on Vercel utilizing stateless API routes for heavy NLP computations and authentication logic.
- **Data Layer:** Local JSON for rapid question retrieval and XLSX (Excel) files for logging user authentication states.

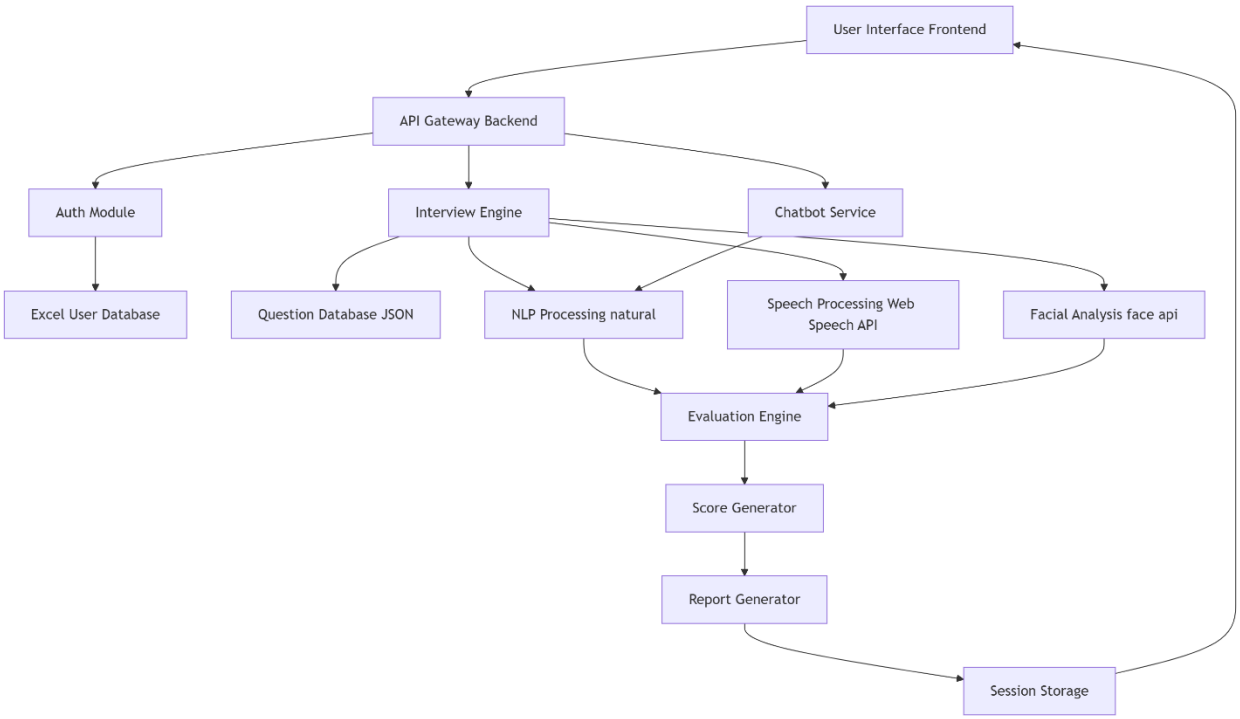

Figure 1 Block diagram

Features built, language used

- **Features:** Real-time facial micro-expression analysis (Confidence/Nervousness), live speech-to-text transcription, semantic answer evaluation (keyword coverage and similarity), background cheat detection (tab-switching), and an integrated AI Interview Assistant chatbot.
- **Languages & Core Tech:** Vanilla JavaScript (ES6+), HTML5, CSS3, Node.js.
- **Libraries:** face-api.js (Computer Vision), natural (NLP Engine), xlsx (Data parsing).

Proposed Methodology

The platform captures continuous user audio and video during an active mock interview session. To ensure privacy and reduce server latency, facial landmark detection is processed entirely on the client side using the user's browser. Simultaneously, the Web Speech API transcribes spoken responses. Upon submission, the transcript is sent to the serverless backend, where the NLP engine tokenizes, stems, and evaluates the text against a curated database of ideal answers.

Algorithm/Description of the Work

1. Initialize the interview session and fetch subject-specific questions.
2. Begin parallel hardware tracking: Web Audio API tracks decibel levels (for Words Per Minute and silence detection) while face-api.js extracts 68-point facial landmarks to classify emotional states.
3. Upon answer submission, strip stop-words and apply stemming to the text transcript.
4. Calculate Term Frequency-Inverse Document Frequency (TF-IDF) to weigh the importance of specific technical jargon.
5. Compute cosine similarity between the user's processed transcript and the database's ideal answer.
6. Aggregate the data to output Technical, Communication, and Behavioral scores out of 10.

Division of the work among students

- **Aman Kumar Singh (Lead Frontend Developer):** Designed the complete Glassmorphism UI/UX architecture, set up the Vite build environment, integrated the Web Speech API, and managed the React SPA state.
- **Saksham Tripathi (Lead AI Integration Engineer):** Implemented the face-api.js module, tuned facial detection thresholds, captured confidence metrics, and built the visibility-based cheat detection system.
- **Shubhankar Kumar (Backend and Data Engineer):** Configured the Node.js/Vercel serverless backend, developed the mathematical NLP algorithms using the natural library, and constructed the XLSX authentication system.

Results

The project resulted in a fully functional, zero-latency web application capable of comprehensively simulating technical interviews. The NLP engine successfully grades technical answers based on semantic similarity rather than rigid exact matching. Furthermore, the real-time behavioral tracking accurately identifies nervousness and engagement without inducing server lag, maintaining high privacy by keeping video processing entirely local.

Conclusion

The AI Interview Analyzer effectively bridges the gap between raw technical knowledge and practical interview execution. By synthesizing modern serverless web technologies with client-side Artificial Intelligence, it provides candidates with a highly objective, multi-modal tutoring platform to refine their communication skills and mitigate interview anxiety.